

Testing

Test de unidad

Test que asegura que la unidad mínima de nuestro programa funciona correctamente, y aislada de otras unidades.

- En nuestro caso, la unidad de teste es el método.

Testear un método es confirmar que el mismo acepta el rango esperado de entradas, y que retorna el valor esperado en cada caso.

Tengo en cuenta:

- Parametros.
- Estado del objeto antes de ejecutar el método.
- Objeto que retorna el método.
- Estado del objeto al conducir la ejecución del método.

Test automatizados

Se utiliza en software para guiar la ejecución de los test y controlar los resultados.

Requiere que diseñemos, programemos y mantengamos programas "test".

- En nuestro caso esos programas serán objetos.

Suele basarse en herramientas que resuelven gran parte del trabajo.

Una vez escritos, los puedo reproducir a costo mínimo, cuando quiera.

Los test son "parte del software" (y un indicador de su calidad).

jUnit

jUnit es una herramienta para simplificar la creación de test de unidad y automatizar su ejecución y reporte.

Ayuda a escribir/programar test utiles.

Cada test se ejecuta independientemente de otros(aislados).

jUnit detecta, recolecta y reporta errores y problemas.

xUnit es su nombre genérico. (lo mismo para otros lenguajes).

Anotmia de un test suite jUnit

- Una clase de test por cada clase a testear.

- Un método que prepara lo que necesitan los test (el fixture).

- Uno o varios métodos de test por cada método a testear.

- Un método que limpia lo que se preparó si es necesario.

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

public class PersonaTest {

    Persona james;

    @BeforeEach
    void setUp() throws Exception {
        james = new Persona();
        james.setApellido("Glosing");
        james.setNombre("James");
    }

    @Test
    public void testNombreCompleto() {
        assertEquals("Glosing, James",
            james.getNombreCompleto());
    }
}
```

Un test es un método que verifica automáticamente que una parte del programa funciona como debería.

La idea general es:

1. Crear objetos.
2. Ejecutar una acción.
3. Verificar el resultado esperado.

Ejemplo:

```
CajaDeAhorro caja = new CajaDeAhorro();  
caja.depositar(1000);
```

```
assertEquals(980, caja.getSaldo());
```

Estoy diciendo después de depositar 1000, espero que el saldo sea 980.

Estructura típica de un test

Arrange (preparación) – Act (acción) – Assert (verificación)

```
@Test  
void testDepositar() {  
    // Arrange (preparación)  
    CajaDeAhorro caja = new CajaDeAhorro();  
  
    // Act (acción)  
    caja.depositar(1000);  
  
    // Assert (verificación)  
    assertEquals(980, caja.getSaldo());  
}
```

Preparar -> Ejecutar -> Verificar

@Test

Indica que el método es un test.

@BeforeEach

Sirve para evitar repetir código.

Assertions

Son las verificaciones

assertEquals: verifica igualdad.

assertTrue: verifica que algo sea verdadero.

assertFalse: verifica que algo sea falso.

assertNull: verifica si algo es null.

assertNotNull: verifica si algo no es null.